CoreML: Implementing the Transformer and the Variations.

Bhuvanesh Reddy

May 6, 2026

Abstract. I present a from-scratch implementation of the Baseline Transformer architecture along with exploring variants in the positional encoding and attention heads.

1. Introduction

The Transformer [1] discards recurrence entirely, relying solely on attention mechanisms to model dependencies between positions. This section motivates the architecture and outlines the contributions of this assignment.

Definition — Scaled Dot-Product Attention

Given queries $\boldsymbol{Q} \in \mathbb{R}^{n \times d_k}$, keys $\boldsymbol{K} \in \mathbb{R}^{m \times d_k}$, and values $\boldsymbol{V} \in \mathbb{R}^{m \times d_v}$, attention is defined as

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V}. \quad (1)$$

The $\sqrt{d_k}$ scaling prevents the dot products from entering saturation regions of the softmax when d_k is large.

2. Architecture

2.1 Multi-Head Attention

Rather than computing a single attention function, the model projects $\boldsymbol{Q}$, $\boldsymbol{K}$, $\boldsymbol{V}$ h times with learned linear projections, attends in parallel, then concatenates:

$$\text{MHA}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\,\boldsymbol{W}^O \quad (2)$$

where $\text{head}_i = \text{Attention}(\boldsymbol{Q}\boldsymbol{W}_i^Q, \boldsymbol{K}\boldsymbol{W}_i^K, \boldsymbol{V}\boldsymbol{W}_i^V)$.

Theorem — Expressivity of MHA

Multi-head attention with h heads and $d_{\text{model}} = h\,d_k$ parameters is strictly more expressive than single-head attention with the same parameter budget, as each head can attend to a distinct subspace of the representation.

2.2 Position-wise Feed-Forward Network

Each encoder/decoder layer includes a two-layer FFN applied identically to each position:

$$\text{FFN}(\boldsymbol{x}) = \max(0,\, \boldsymbol{x}\boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2 \quad (3)$$

Remark: Dimensionality

The inner dimension is typically $d_{ff} = 4\,d_{\text{model}}$. In the base model [1], $d_{\text{model}} = 512$ and $d_{ff} = 2048$.

2.3 Positional Encoding

Since attention is permutation-equivariant, position information must be injected explicitly. Sinusoidal encodings are given by:

$$\text{PE}_{(pos,\,2i)} = \sin\left(pos/10000^{2i/d}\right) \quad (4)$$

$$\text{PE}_{(pos,\,2i+1)} = \cos\left(pos/10000^{2i/d}\right) \quad (5)$$

Important — Learned vs. Sinusoidal

Learned positional embeddings and sinusoidal encodings achieve comparable performance; however, sinusoidal encodings generalise to sequence lengths unseen during training.

3. Implementation

3.1 Attention Module

The core attention function in PyTorch:

Python - Scaled Dot-Product Attention

```python
import torch
import torch.nn.functional as F
import math

def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    Q, K, V : (batch, heads, seq, d_k)
    """
    d_k = Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V), weights
```

3.2 Layer Normalisation

Residual connections are followed by layer normalisation:

$$\boldsymbol{y} = \mathrm{LayerNorm}(\boldsymbol{x} + \mathrm{Sublayer}(\boldsymbol{x})) \tag{6}$$

Remark: Pre-LN vs. Post-LN

Modern implementations (GPT, LLaMA) typically apply normalisation *before* the sublayer (*Pre-LN*), which improves gradient flow and stabilises training without a learning rate warm-up.

4. Training Dynamics

4.1 Learning Rate Schedule

The original Transformer uses a warm-up schedule:

$$\alpha = d_{\mathrm{model}}^{-0.5} \cdot \min\!\left(step^{-0.5},\; step \cdot warmup^{-1.5}\right) \tag{7}$$

Important — Warm-up Critical

Without the linear warm-up phase the model often diverges in early training. A typical value is $warmup = 4000$ steps.

4.2 Regularisation

Three regularisation strategies are employed:

1. **Dropout** applied to attention weights and sublayer outputs ($p_{\mathrm{drop}} = 0.1$).
2. **Label smoothing** with $\varepsilon_{ls} = 0.1$, which hurts perplexity but improves BLEU.
3. **Weight decay** ($\lambda = 10^{-4}$) on all non-bias parameters.

5. Experimental Results

5.1 Ablation: Number of Heads

Heads (h)	Val. Loss	BLEU	Params
1	3.42	18.1	37.4M
4	2.91	23.7	37.4M
8	2.78	25.3	37.4M
16	2.81	24.9	37.4M

Table 1. Effect of the number of attention heads (base model, $d_{\mathrm{model}} = 512$).

Eight heads yields the best BLEU score; increasing to 16 slightly degrades performance, consistent with findings in [2].

6. Conclusion

We have implemented and analysed the Transformer architecture from scratch, covering attention, positional encodings, layer normalisation, and training dynamics. Ablation results confirm the importance of multi-head attention and proper learning rate scheduling. Future work will explore relative positional encodings and sparse attention variants.

References

[1] A. Vaswani et al., "Attention is all you need," *NeurIPS*, 2017.

[2] E. Voita et al., "Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned," *ACL*, 2019.